RMIT UNIVERSITY

School of Computing Technologies

RMIT Society
Cloud Solution Architecture

Assessment 3 — Cloud Computing

Course code and name: `<course code and course name>`
Student name: `<student name>`
Student ID: `<student ID>`
Tutor: `<tutor name>`
Submission date: `<submission date>`
Report version: 0.1 — architecture-report draft
Architecture snapshot: `<architecture snapshot date>`

This boilerplate cover intentionally contains editable placeholders. Replace the student and assessment details, then remove placeholder styling before submission.

Contents

1 Project links and summary

Item	Final value
Live application	`<browser-verified live URL>`
Source repository	`https://github.com/klenathan/ASM3`
Editable architecture sources	`docs/architecture/report/figures/`; existing infrastructure reference: `infras/rmit_society_aws_architecture.py`
Public dataset	No public dataset; deterministic user-generated demonstration data.
Deployment environment	AWS Academy Learner Lab, `us-east-1`.

RMIT Society is an RMIT-only discussion forum that gives students a common place to discover communities, publish threads, comment, vote, attach media, attach a verified place, and report content. A *Society* is a community and a *Thread* is a post in a society. The application distinguishes students, society-scoped moderators, and the globally elevated `system_admin` role. Its cloud objective is not simply to host a web application: a visible user operation must invoke the appropriate supporting service across web hosting, networking, containers, compute, storage, transactional persistence, and, when enabled and demonstrated, asynchronous content analysis and analytics.

The core demonstration path is a React/Vite single-page application (SPA) served by Amplify Hosting. Its `/api/*` requests are rewritten to an API Gateway HTTP API, which proxies to a domain-oriented modular-monolith backend running as an ECS service on one EC2 container instance. PostgreSQL on Amazon RDS is the authoritative transactional store; S3 retains media binaries while RDS retains only their metadata and object keys. This report separates deployed demonstration evidence from feature-gated optional pipelines. It is therefore an architecture explanation for the assessed environment, not an implementation manual or a claim that every provisionable service is active. AWS component characteristics cited in this report are documented by AWS [1, 2, 3, 4, 5].

2 Introduction

2.1 Motivation and product view

Student discussion is commonly fragmented across broad social channels, course Q&A tools, and informal groups. RMIT Society narrows that context to an institution-specific forum, while preserving familiar community discussion patterns. Registration is restricted by a configurable approved RMIT email-domain policy. A student can join societies, create and manage their own discussions, vote, attach media, associate one server-verified location, and report content. A moderator has authority only through an active membership in the society concerned. A `system_admin` is the only globally elevated role.

The implementation is deliberately a modular monolith rather than a collection of independently deployed microservices. A single backend keeps transaction handling, demonstration operations, and cost proportionate to the coursework deployment; explicit domain boundaries retain a future extraction path if usage justifies it. Browser route guards improve navigation but do not authorize an operation: application services enforce ownership, membership, moderator scope, suspension, and platform-administrator policy.

2.2 Beneficiaries

Beneficiary	Benefit
Students	RMIT-specific discussion, discovery, and collaboration in relevant societies.
Society moderators	A society-scoped report queue, membership controls, and traceable moderation decisions.
System administrators	User and society governance plus interpretable aggregate activity reporting when the analytics pipeline is enabled.

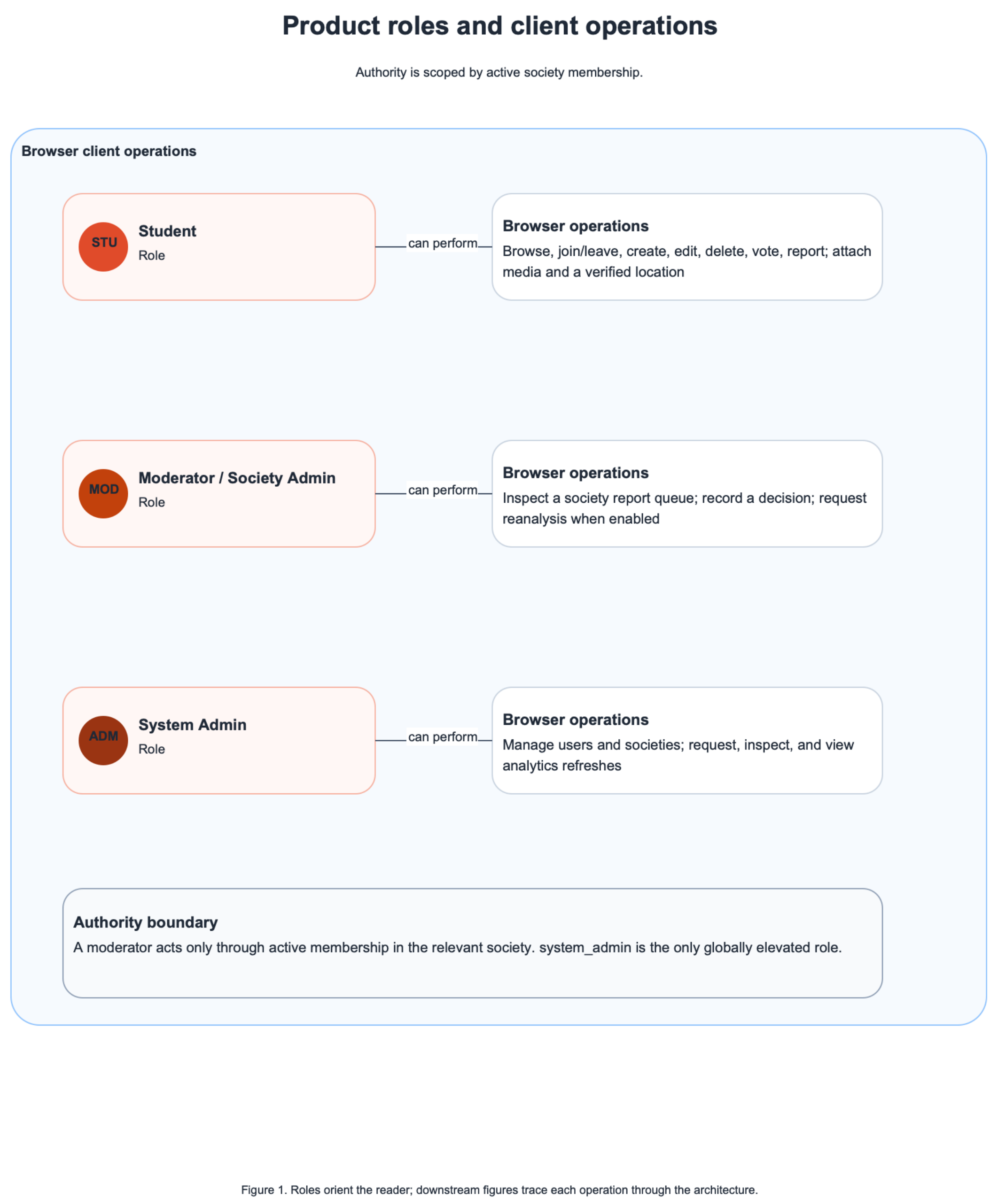

Figure 1: Product roles and client operations. The diagram orients the reader; the operation coverage table, Figure 3, and the detailed sequences in Appendix A provide the downstream invocation paths.

3 Related work

3.1 Community discussion platforms

Reddit documents communities, voting, and moderator tools as key community mechanisms [6]. Discourse documents structured discussion and moderation-oriented trust and administration capabilities [7]. RMIT Society reuses the recognisable society/thread/vote model, but its access boundary and moderator authority are purposefully constrained to the RMIT context rather than a general-purpose public community.

3.2 Institution-oriented communication platforms

Piazza presents a course-oriented question-and-answer communication model [8]. CampusGroups documents institution and organisation membership workflows [9]. These products demonstrate the value of institution-aware participation, but their documented emphasis differs from a discussion-first society forum with threads, media, reporting, and society-scoped moderation.

3.3 Design distinction

RMIT Society combines configurable institutional registration, membership-derived moderator authority, direct S3 media upload with relational metadata, server-verified Mapbox place attachment, and optional asynchronous content-analysis and aggregate-analytics workflows. This is a scope distinction, not a superiority claim.

Table 1: Related-work comparison.

Related work	**Demonstrated pattern**	**Limitation relative to this project**	**RMIT Society response**
Reddit	Community structure, voting, volunteer moderation.	No RMIT-specific access boundary or membership-derived authority.	Restrict audience and derive moderation power from society membership.
Discourse	Structured discussions and moderation/trust concepts.	General-purpose community deployment.	Focus on RMIT societies and demonstrable AWS-backed workflows.
Piazza	Institution-oriented academic discussions.	Course/Q&A emphasis.	Support broader society threads, media, reports, and moderation.
CampusGroups	Student organisation and institution membership.	Organisation-management orientation.	Provide a discussion-first community model.

4 System architecture

Deployment-status note: replace every provisional statement in this report with evidence from the final demonstration environment before submission.

4.1 Architecture reading guide

Solid arrows in the architecture figures are synchronous requests or responses. Dashed arrows are asynchronous messages, scheduled triggers, or orchestration transitions. Numbered arrows express their order. Every diagram separates the browser, the AWS Cloud in `us-east-1`, and third-party services. Orange-tinted boxes are optional and must only be presented as live assessment evidence after the complete path is deployed and demonstrated.

Table 2: Client-operation coverage.

Client operation	**Actor**	**Architecture figure**
Register, sign in, refresh session, sign out; browse societies, feeds, threads, and comments	Student or visitor	Figure 3
Join or leave a society; create, edit, or delete thread/comment; vote	Student	Figure 3
Request, upload, complete, and attach media	Student	Figure A.1
Search, select, and persist a verified location	Student	Figure A.1
Submit report; inspect queue; record decision; request reanalysis	Student or moderator	Figure A.2
Request, monitor, and view analytics refresh	System administrator	Figure A.3

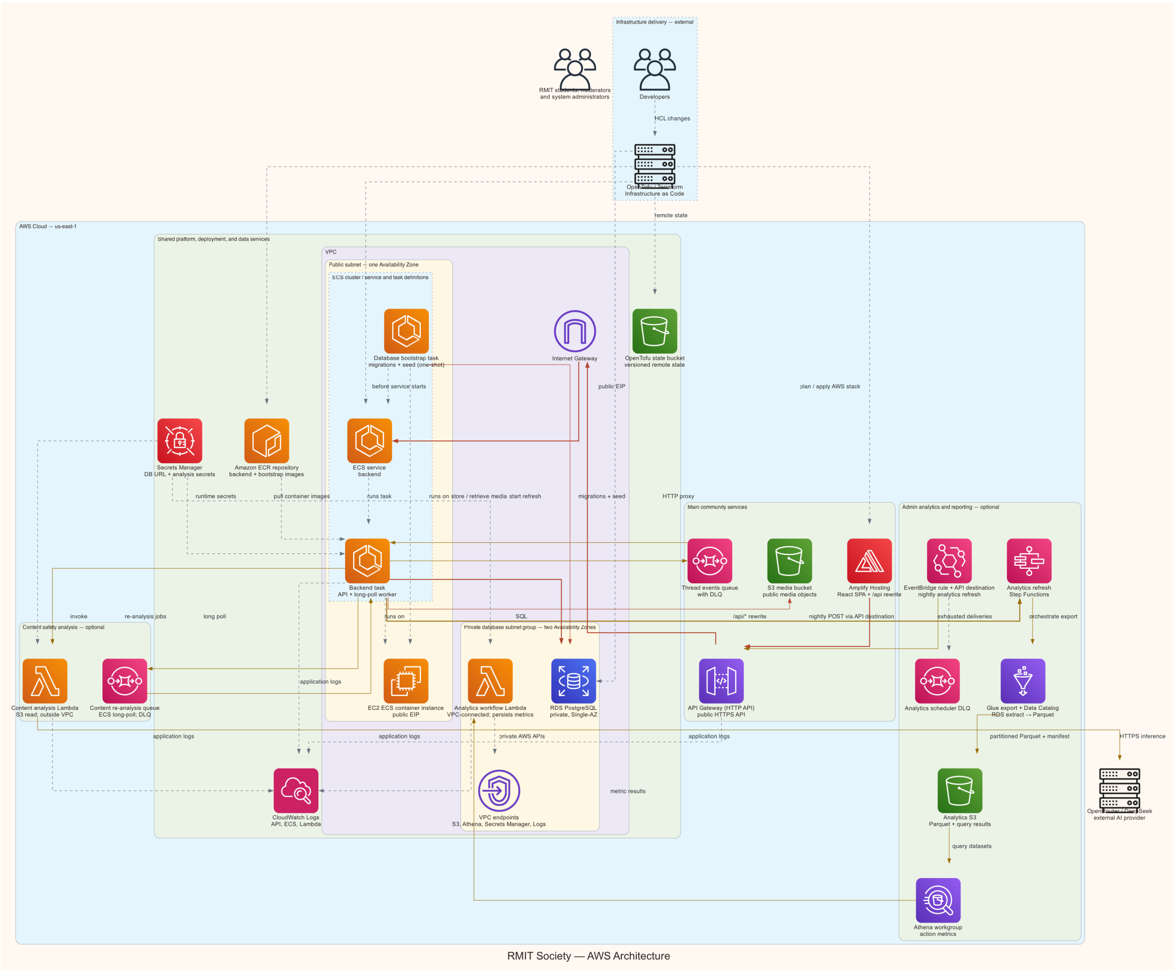

Figure 2: Deployment and component overview. The primary path is the active topology documented in infrastructure source; content analysis and analytics are feature-gated and must be status-labelled in the final report.

4.2 Core authentication and community request flow

Figure 3 covers synchronous community operations. The SPA invokes an application or API operation; Amplify rewrites `/api/*` to API Gateway; API Gateway proxies it to ECS; and the backend applies its dependency flow: route, controller, application service, domain policy, repository port, and Drizzle repository. The repository reads or writes RDS PostgreSQL, using a transaction when a use case requires one, before the response returns through API Gateway. This sequence makes the backend a domain-oriented modular monolith, not independently deployed microservices.

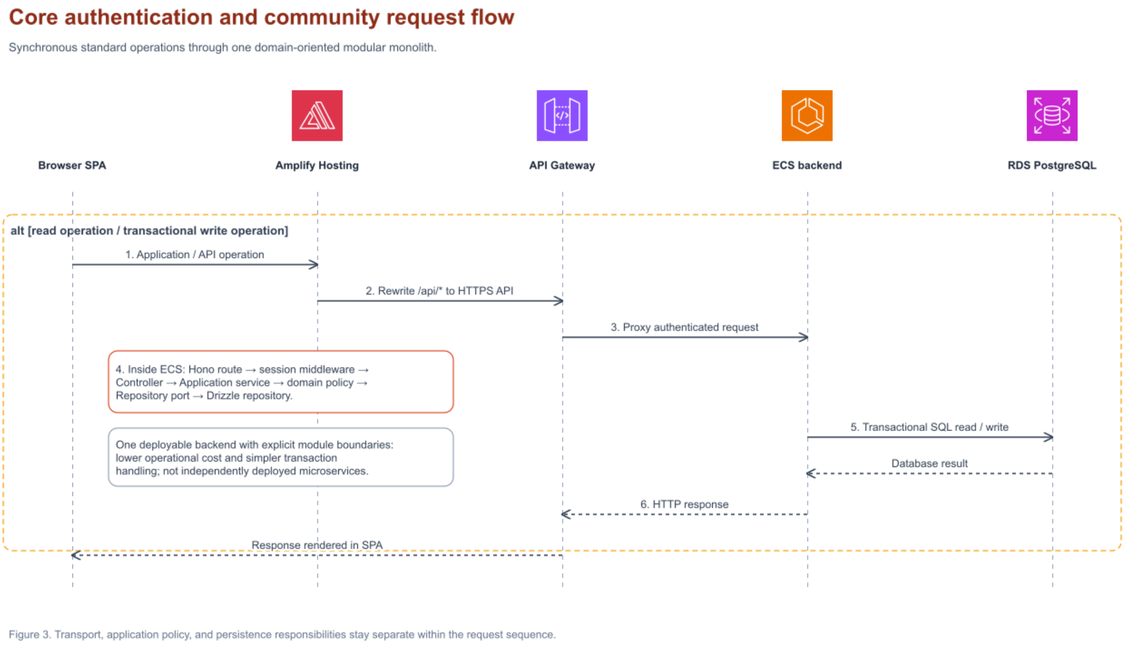

Figure 3: Core authentication and community request flow. The internal ECS dependency flow shows that transport, application policy, and persistence responsibilities are distinct.

4.3 Media upload and verified location flow

The media lane in Appendix Figure A.1 (click to view) keeps binary transfer off the application backend. The backend authorizes the request and creates pending metadata in RDS, then returns a presigned S3 upload URL. The browser uploads the binary directly to S3 and confirms completion through the API. The backend changes the media state to ready and attaches it to the thread. RDS retains metadata and the object key, never the media binary or presigned URL.

The location lane is intentionally server verified. The place interface submits the selected Mapbox identifier, not arbitrary client coordinates. The ECS places adapter retrieves and verifies the selected place before persistence, then stores the verified name, Mapbox identifier, place type, latitude, and longitude with the thread. Mapbox documents its Search service and place-retrieval interfaces [10].

4.4 Moderation and optional content analysis

A student report is persisted through the standard SPA, API Gateway, ECS, and RDS path. A moderator decision additionally verifies that the principal has an active moderator membership for the same society. This is different from the global `system_admin` role. Appendix Figure A.2 (click to view) depicts the content-reanalysis route as an *implemented optional pipeline*: it must remain clearly marked as non-live until its SQS, worker, Lambda, OpenRouter, storage-access, persistence, and visible-result path are exercised. OpenRouter documents its API as the inference integration boundary [11].

4.5 Optional analytics refresh and reporting

Appendix Figure A.3 (click to view) describes the analytics v2 operation when enabled: a system administrator requests an inclusive-start/exclusive-end UTC date-range refresh, or EventBridge invokes the authenticated scheduler endpoint. ECS creates one durable run in RDS and starts a Step Functions execution with range-scoped input. Step Functions owns waiting, retry, timeout, and fan-in rather than ECS polling Glue or Athena. Glue exports action events, memberships, and votes to private S3 and catalogs Parquet external tables. Athena runs activity, current-state, and reconciliation queries in parallel. The workflow Lambda validates the results and persists metric rows to RDS, which the Admin Center reads through ECS.

Actor identities in action events are HMAC pseudonyms rather than direct user identifiers. The entire analytics path is feature-gated in the documented infrastructure. It is not live assessment evidence until the refresh request, durable status, Glue output, Athena query, persisted metric, and visible table/chart have all been demonstrated. AWS documents the orchestration, ETL, query, and scheduling services used by this optional path [12, 13, 14, 15].

5 System descriptions

The component table links every service to a browser operation or automatic service-to-service invocation. Final submission must remove or relabel any row whose component is unavailable in the demonstration environment.

Table 3: Component responsibilities and invocation paths.

Category	Component	Purpose and invocation
Client	React/Vite SPA	Browser interface for student, moderator, and administrator operations.
Hosting	Amplify Hosting	Serves the SPA and rewrites `/api/*` requests to API Gateway.
Networking	API Gateway HTTP API	Public HTTPS entry point and proxy to the ECS backend.
Containers / compute	ECS on EC2	Runs backend and worker containers on a low-cost single container instance.
Image registry	ECR	Stores backend and database-bootstrap container images used by ECS.
Database	RDS for PostgreSQL	Authoritative transactional store and persisted aggregate metric store.
Storage	S3	Stores user-uploaded media; stores private analytics export artefacts when the analytics path is enabled.
Messaging	SQS	Delivers moderator-requested content reanalysis to the worker when enabled.
Serverless	Lambda	Runs the content-analysis adapter and analytics result-persistence handler when their respective paths are enabled.
Orchestration	Step Functions	Coordinates analytics export/query execution, retry, waiting, and fan-in when enabled.
Analytics	Glue, Athena, EventBridge	Export/catalog data, query analytics datasets, and trigger scheduled refreshes when the pipeline is enabled.
Runtime support	Secrets Manager, CloudWatch	Supplies runtime secrets without source-control exposure and retains application/service logs.
Third party	Mapbox, OpenRouter	Mapbox verifies selected places; OpenRouter is the enabled-only content-analysis inference endpoint.

6 Datasets, data structures, and APIs

6.1 Data origin and classification

Data source	Storage	Privacy and retention position
Accounts, profiles, societies, memberships, threads, comments, votes, reports	RDS PostgreSQL	Access-controlled by session, ownership, membership, and role policy.
Media objects	S3	Store binary objects in S3; retain object keys and metadata only in RDS.
Location snapshots	Thread fields in RDS	Persist only server-verified place metadata relevant to the thread.
Action events	RDS, then private analytics S3 during refresh	Use HMAC actor pseudonyms and a bounded raw-event retention period.
Aggregate analytics metrics	`analytics_action_metrics` in RDS	Administrative aggregate reporting.
Content-analysis data	RDS/S3/Lambda workflow when enabled	Never publish credentials, private URLs, or secret values.

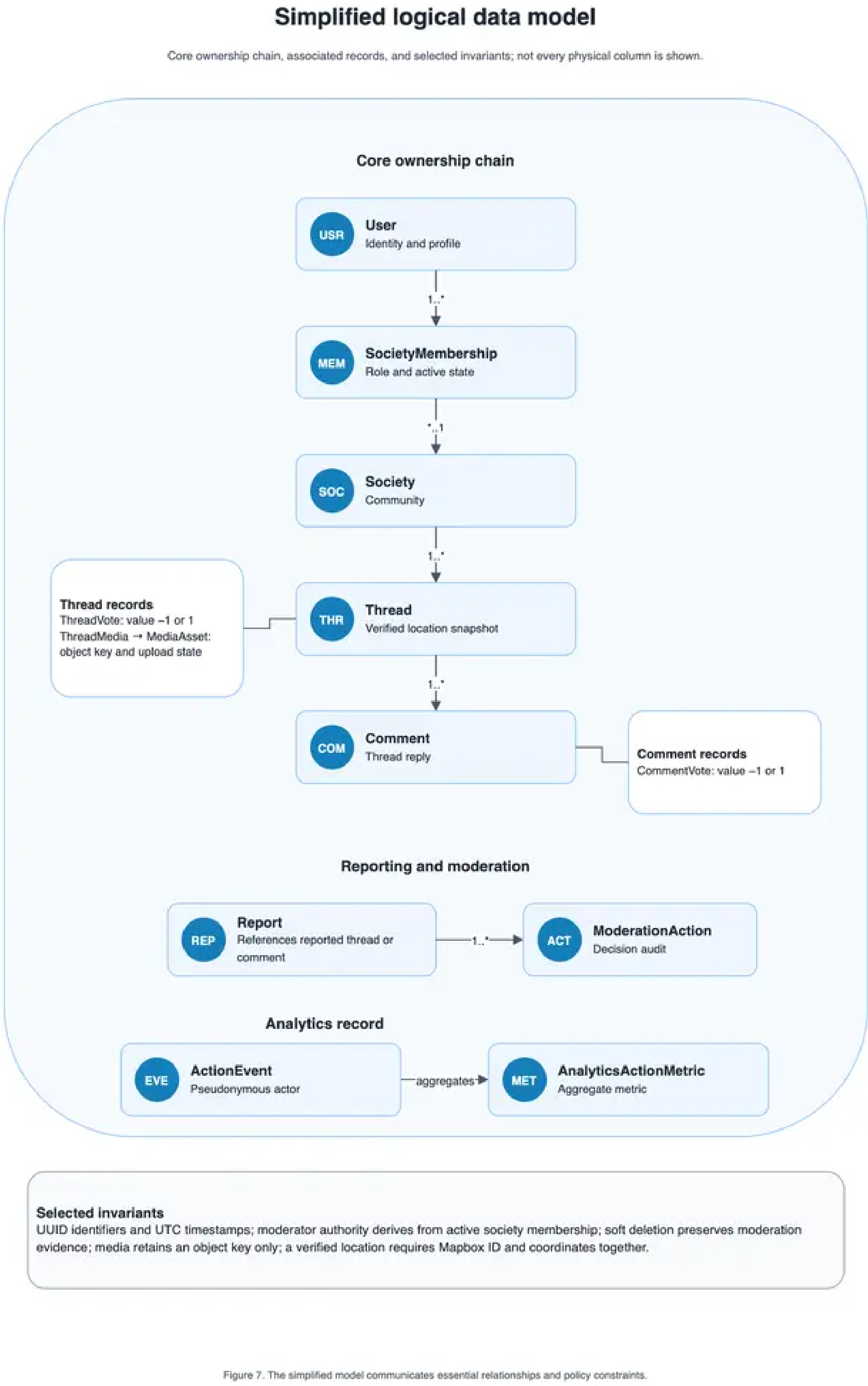

Figure 4: Simplified logical data model. It communicates relationships and invariants rather than every physical column.

6.2 Logical data invariants

Identifiers are application-generated UUIDs and timestamps use UTC. Vote values are constrained to -1 or 1. Moderator authority is derived from active society membership. Soft deletion and status transitions preserve moderation evidence. Media records retain an object key but no presigned or delivery URL. A persisted location requires the Mapbox identifier and its verified coordinate fields together.

6.3 API catalogue and boundaries

API group	Representative operations	Consumer
Authentication	Register, sign in, refresh session, sign out.	SPA
Societies	Browse, inspect, join/leave, manage membership.	SPA
Discussions	Create/list/read/update/delete threads and comments; vote.	SPA
Media	Request upload, confirm upload, attach asset.	SPA and direct S3 upload
Moderation	Submit report, inspect queue, decide, request reanalysis.	SPA
Places	Search, retrieve, reverse geocode.	SPA through ECS
Analytics v2	Query metrics; create, cancel, inspect refresh.	Admin Center
Scheduled analytics	Start scheduled refresh.	EventBridge only

The browser communicates with the ECS backend through API Gateway. It does not directly access RDS, Secrets Manager, Glue, Athena, or other private services. The Mapbox and OpenRouter contracts are mediated by backend or Lambda adapters so that credentials and verification policies remain server-side.

7 Evidence register and finalisation checklist

The following register is intentionally unfilled. It prevents the final report from promoting a planned component to deployed evidence.

Table 4: Evidence register to complete during deployment rehearsal.

Claimed capability	Evidence to capture, redacted	Report use
SPA hosting and API routing	Browser screenshot and successful authenticated workflow.	Links, Figure 2, demo
ECS/EC2 backend execution	ECS service/task state and CloudWatch evidence.	Figure 2, Table 3
RDS persistence	Visible create/read/update workflow and redacted task/database evidence.	Figures 3–A.2
S3 media path	Upload, retrieval, and browser-visible attachment.	Figure A.1
Mapbox location verification	Create and display a thread with one verified location.	Figure A.1

Claimed capability	Evidence to capture, redacted	Report use
Moderator boundary	Student report and moderator decision within one society.	Figure A.2
Content analysis	Enqueue, process, persist, and display result, if enabled.	Figure A.2
Analytics pipeline	Refresh, status, Glue output, Athena queries, persisted metric, displayed table/chart, if enabled.	Figure A.3

Before export, replace the cover and link placeholders; verify the live URL in a browser; confirm every claimed AWS service is provisioned and exercised; ensure every client operation appears in the coverage table and a flow figure; label feature-gated components accurately; include no secrets, private URLs, full ARNs, account identifiers, or credential material; and copy the rendered figure PDFs to the submission package's `doc_images/` directory.

A Detailed operation sequences

The main architecture section links to these full-width operation sequences. They are placed here to keep the main narrative concise while retaining the complete invocation paths.

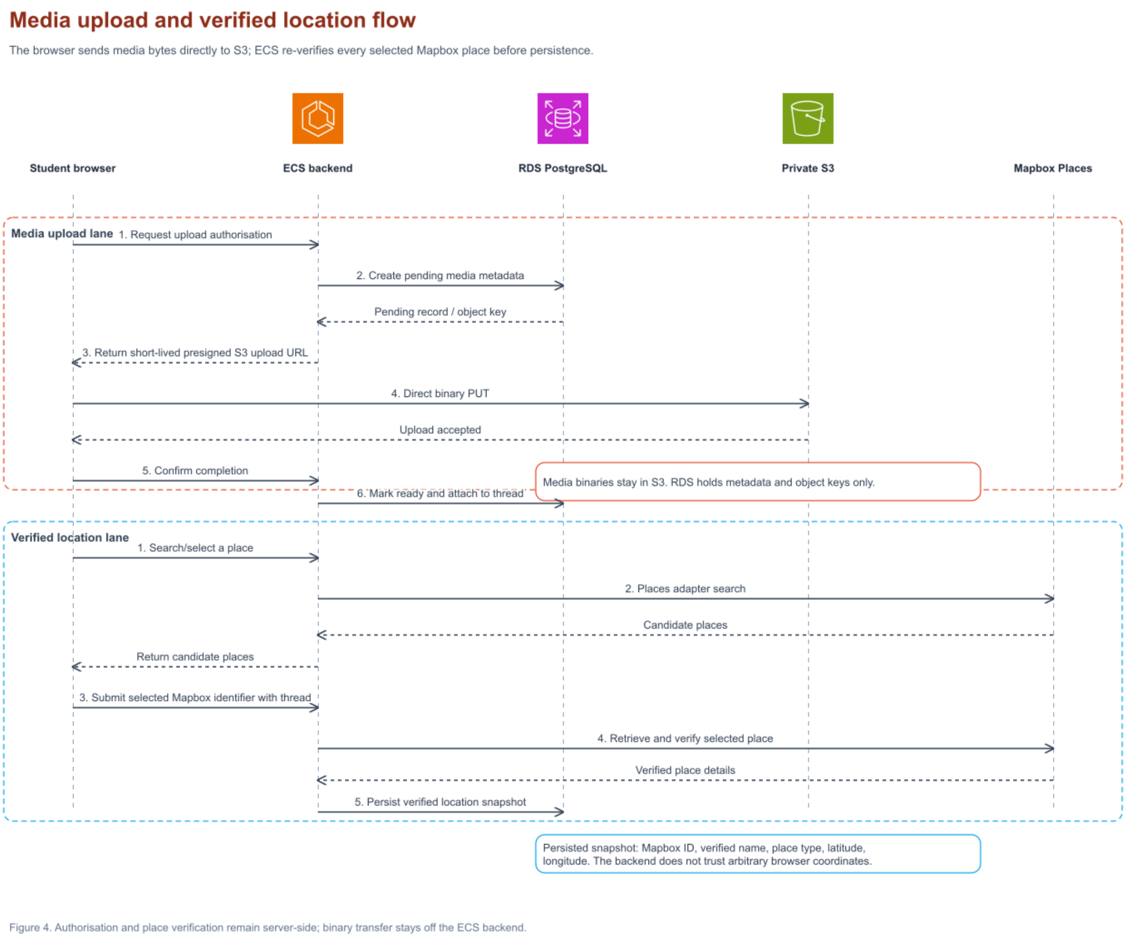

Figure A.1: Two-lane media-upload and verified-location flow. The backend controls authorization and persistence; the browser sends media bytes directly to S3 and location data is verified against Mapbox.

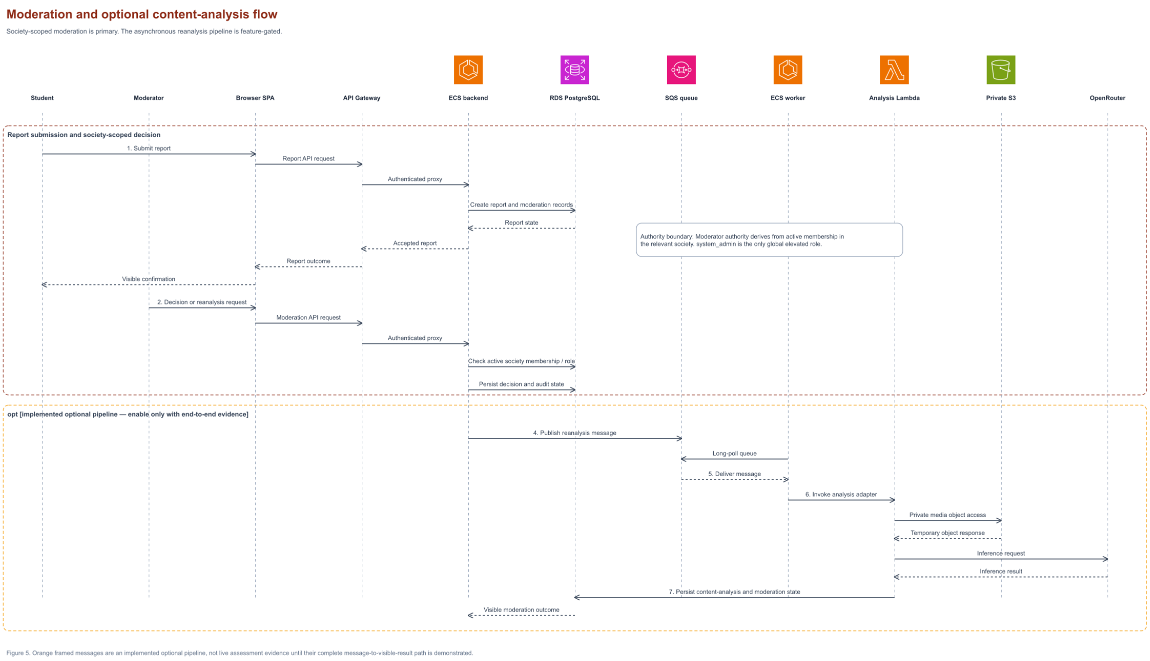

Figure A.2: Moderation and optional content-analysis flow. Orange elements are feature-gated; do not claim them as deployed evidence without an end-to-end observation.

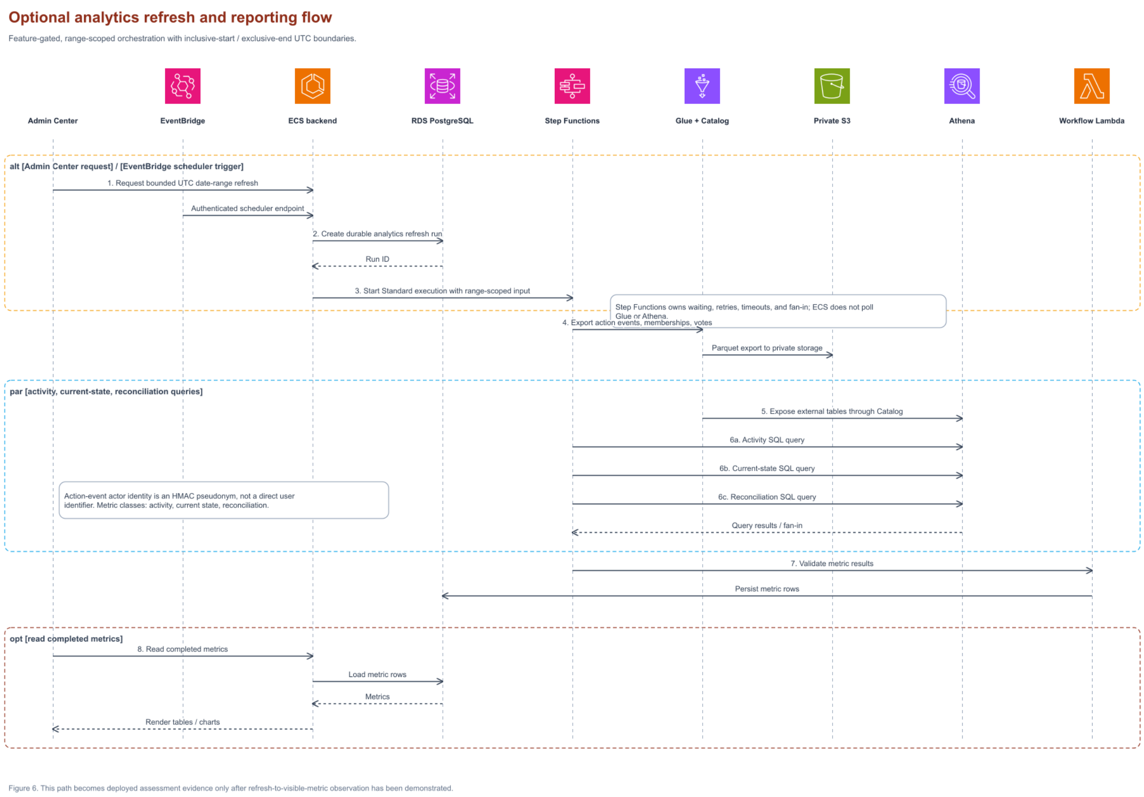

Figure A.3: Optional analytics refresh and reporting flow. Dashed arrows represent scheduling and orchestration transitions. The full path must be enabled and observed before it is stated as deployed.

References

[1] Amazon Web Services, "Aws amplify hosting user guide," https://docs.aws.amazon.com/amplify/latest/userguide/welcome.html, 2026, accessed: 2026-09-08.

[2] ——, "What is amazon api gateway?" https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html, 2026, accessed: 2026-09-08.

[3] ——, "What is amazon elastic container service?" https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html, 2026, accessed: 2026-09-08.

[4] ——, "What is amazon rds?" https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html, 2026, accessed: 2026-09-08.

[5] ——, "Amazon s3 user guide," https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html, 2026, accessed: 2026-09-08.

[6] Reddit, Inc., "Reddit help: Communities and moderation," https://support.reddithelp.com/hc/en-us/categories/200073949-Reddit-101, 2026, accessed: 2026-09-08.

[7] Discourse, "Discourse documentation," https://meta.discourse.org/c/documentation/6, 2026, accessed: 2026-09-08.

[8] Piazza Technologies, Inc., "Piazza: The q&a platform for class," https://piazza.com/product, 2026, accessed: 2026-09-08.

[9] CampusGroups, "Campusgroups: Campus engagement platform," https://www.campusgroups.com/, 2026, accessed: 2026-09-08.

[10] Mapbox, "Search api documentation," https://docs.mapbox.com/api/search/, 2026, accessed: 2026-09-08.

[11] OpenRouter, "Openrouter api documentation," https://openrouter.ai/docs/api-reference/overview, 2026, accessed: 2026-09-08.

[12] Amazon Web Services, "What is step functions?" https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html, 2026, accessed: 2026-09-08.

[13] ——, "What is aws glue?" https://docs.aws.amazon.com/glue/latest/dg/what-is-glue.html, 2026, accessed: 2026-09-08.

[14] ——, "What is amazon athena?" https://docs.aws.amazon.com/athena/latest/ug/what-is.html, 2026, accessed: 2026-09-08.

[15] ——, "What is amazon eventbridge?" https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html, 2026, accessed: 2026-09-08.